



Module _02_getting_started

[egg::tutorials](#)

Module _02_getting_started



Search



Summary

My first egg

This tutorial is aimed at getting you up and running with `egg`, even if you have little Rust experience. If you haven't heard about e-graphs, you may want to read the [background tutorial](#). If you do have prior Rust experience, you may want to skim around in this section.

Getting started with Rust

[Rust](#) is one of the reasons why `egg` is fast (systems programming + optimizing compiler) and flexible (generics and traits).

The Rust folks have put together a great, free [book](#) for learning Rust, and there are a bunch of other fantastic resources collected on the “[Learn](#)” [page of the Rust site](#). This tutorial is no replacement for those, but instead it aims to get you up and running as fast as possible.

First, [install](#) Rust and let's [create a project](#) with `cargo`, Rust's package management and build tool: `cargo new my-first-egg`.¹

Now we can add `egg` as a project dependency by adding a line to `Cargo.toml`:

```
[dependencies]
egg = "0.11.0"
```

All of the code samples below work, but you'll have to use the relevant types. You can just bring them all in with a `use egg::*;` at the top of your file.

Now you're speaking my [Language](#)

[EGraphs](#) (and almost everything else in this crate) are parameterized over the [Language](#) given by the user. While `egg` supports the ability easily create your own [Language](#), we will instead start with the provided [SymbolLang](#).

`Language` is a trait, and values of types that implement `Language` are e-nodes. An e-node may have any number of children, which are `Ids`. An `Id` is basically just a number that egg uses to coordinate what children an e-node is associated with. In an `EGraph`, e-node children refer to e-classes. In a `RecExpr` (egg's version of a plain old expression), e-node children refer to other e-nodes in that `RecExpr`.

Most `Language`s, including `SymbolLang`, can be parsed and pretty-printed. That means that `RecExpr`s in those languages implement the `FromStr` and `Display` traits from the Rust standard library.

```
// Since parsing can return an error, `unwrap` just panics if the result is an error
let my_expression: RecExpr<SymbolLang> = "(foo a b)".parse().unwrap();
println!("this is my expression {}", my_expression);

// let's try to create an e-node, but hmmm, what do I put as the children?
let my_enode = SymbolLang::new("bar", vec![]);
```

Some e-nodes are just constants and have no children (also called leaves). But it's intentionally kind of awkward to create e-nodes with children in isolation, since you would have to add meaningless `Ids` as children. The way to make meaningful `Ids` is by adding e-nodes to either an `EGraph` or a `RecExpr`:

```

let mut expr = RecExpr::default();
let a = expr.add(SymbolLang::leaf("a"));
let b = expr.add(SymbolLang::leaf("b"));
let foo = expr.add(SymbolLang::new("foo", vec![a, b]));

// we can do the same thing with an EGraph
let mut egraph: EGraph<SymbolLang, ()> = Default::default();
let a = egraph.add(SymbolLang::leaf("a"));
let b = egraph.add(SymbolLang::leaf("b"));
let foo = egraph.add(SymbolLang::new("foo", vec![a, b]));

// we can also add RecExprs to an egraph
let foo2 = egraph.add_expr(&expr);
// note that if you add the same thing to an e-graph twice, you'll get
assert_eq!(foo, foo2);

```

Searching an EGraph with Patterns

Now that we can add stuff to an [EGraph](#), let's see if we can find it. We'll use a [Pattern](#), which implements the [Searcher](#) trait, to search the e-graph for matches:

```

// let's make an e-graph
let mut egraph: EGraph<SymbolLang, ()> = Default::default();
let a = egraph.add(SymbolLang::leaf("a"));
let b = egraph.add(SymbolLang::leaf("b"));
let foo = egraph.add(SymbolLang::new("foo", vec![a, b]));

// rebuild the e-graph since we modified it
egraph.rebuild();

// we can make Patterns by parsing, similar to RecExprs
// names preceded by ? are parsed as Pattern variables and will match
let pat: Pattern<SymbolLang> = "(foo ?x ?x)".parse().unwrap();

// since we use ?x twice, it must match the same thing,
// so this search will return nothing
let matches = pat.search(&egraph);
assert!(matches.is_empty());

egraph.union(a, b);
// recall that rebuild must be called to "see" the effects of adds or
egraph.rebuild();

// now we can find a match since a = b
let matches = pat.search(&egraph);
assert!(!matches.is_empty());

```

Using Runner to make an optimizer

Now that we can make [Patterns](#) and work with [RecExprs](#), we can make an optimizer! We'll use the [rewrite!](#) macro to easily create [Rewrites](#) which consist of a name, left-hand pattern to search for, and right-hand pattern to apply. From there we can use the [Runner](#) API to run egg's equality saturation algorithm. Finally, we can use an [Extractor](#) to get the best result.

```

use egg::{*, rewrite as rw};

let rules: &[Rewrite<SymbolLang, ()>] = &[
    rw!("commute-add"; "(+ ?x ?y)" => "(+ ?y ?x)"),
    rw!("commute-mul"; "(* ?x ?y)" => "(* ?y ?x)"),

    rw!("add-0"; "(+ ?x 0)" => "?x"),
    rw!("mul-0"; "(* ?x 0)" => "0"),
    rw!("mul-1"; "(* ?x 1)" => "?x"),
];

// While it may look like we are working with numbers,
// SymbolLang stores everything as strings.
// We can make our own Language later to work with other types.
let start = "(+ 0 (* 1 a)).parse().unwrap();

// That's it! We can run equality saturation now.
let runner = Runner::default().with_expr(&start).run(rules);

// Extractors can take a user-defined cost function,
// we'll use the egg-provided AstSize for now
let extractor = Extractor::new(&runner.egraph, AstSize);

// We want to extract the best expression represented in the
// same e-class as our initial expression, not from the whole e-graph.
// Luckily the runner stores the eclass Id where we put the initial ex
let (best_cost, best_expr) = extractor.find_best(runner.roots[0]);

// we found the best thing, which is just "a" in this case
assert_eq!(best_expr, "a".parse().unwrap());
assert_eq!(best_cost, 1);

```

-
1. By default cargo will create a binary project. If you are just getting starting with Rust, it might be easier to stick with a binary project, just put all your code in `main`, and use `cargo run`. Library projects (`cargo new --lib my-first-egg`) can be easier to build

on once you want to start writing tests. 